9ThirtyOne

Installation and Setup Guide

For AFROTC detachment administrators · Alpha 0.18

Before you start

Allow about 45 minutes for a first-time setup using Google Drive. You will need the detachment's Google account credentials and permission to create a Google Cloud project on that account. No servers, databases or software licences are required.

Every procedure in this guide has been performed end to end against real Google Drive, on a Mac and an iPhone. Where something is awkward or unfinished, it is written down rather than smoothed over — see *What this version cannot do yet*.

What this guide covers

9ThirtyOne is a web app that runs in a browser and stores every record in a Google Drive folder your detachment owns. There is no vendor, no server and no shared database — which is why setup involves your own Google account rather than an account with us.

1	**Decide where your data will live** — three options, and how to choose
2	**Prepare the Google account** — folder, Cloud project, Drive API, OAuth client
3	**Publish the app** — free hosting, or run it from one computer
4	**Run the setup wizard** — connect the app to your folder
5	**Claim the folder** — the first Google sign-in, and the access setting
6	**Deploy the submission server** — 15 minutes, and it changes what everything below costs
7	**Build the roster** — roles, and adding people by email or CSV
8	**Invite everyone** — one link, or a QR code on a projector
9	**Verify and hand over** — a checklist before first use
–	**Troubleshooting and reference** — common problems, folder layout, backups

What you will need

ITEM	WHY
A new Google account, created for the detachment	Holds every record. Create a fresh one — Part 2a explains why a school or university account is the wrong choice here.
Permission to create a Google Cloud project	Required once, to let the app talk to that account's Drive. A new personal Google account can always do this.
About 45 minutes, once	Most of it is the Google Cloud console in Part 2 and the submission server in Part 6. The app itself takes a few minutes.
Somewhere to publish the app	Any static web host. Several free options are listed in Part 3.
A current browser	Chrome, Edge, Safari or Firefox, on any device.

You are not signing up for anything

9ThirtyOne has no accounts with a vendor, no subscription and no data-processing agreement to sign, because no data ever reaches anyone but you. Everything below sets up *your* Google account to hold *your* detachment's feedback.

1 Decide where your data will live

Pick one. You can change later, but moving records between options is a manual export and import, so it is worth choosing correctly now.

OPTION	BEST FOR	WHAT IT MEANS
Google Drive RECOMMENDED	Any detachment with more than one device	Records live in a Drive folder on your Google account. Works on phones, tablets and computers, and everyone sees the same data. Needs the one-time Google setup in Part 2.
Synced folder	A single office computer	Points at the 9ThirtyOne folder inside Google Drive for Desktop. Google's own client handles syncing. Desktop Chrome or Edge only — no phones.
This device only	Trying the app out	Everything stays in one browser. Nothing syncs, nothing is shared, and clearing site data erases it. Fine for evaluation — export a backup and import it once you move to Drive.

> **Why a browser cannot simply be pointed at a Drive folder path**
>
> Web pages have no access to the file system, and a phone has no synced Drive folder at all. The Google Drive option therefore connects through Google's own Drive service rather than a folder path. This is the reason Part 2 exists.

If you are evaluating the app, choose *This device only*, skip to Part 4, and come back to Part 2 when you are ready to go live. Nothing is lost — Part 8 explains how to carry your trial data across.

2 Prepare the Google account

Do this once, on the Google account that will own the data. About 20 minutes.

2a. Create the detachment's Google account

Create a **new, ordinary Google account** for this — the kind anyone can make at `accounts.google.com`, ending in `@gmail.com`. Do not use a school or university account, and do not use anyone's personal account.

> **Why not the university account**
>
> Exactly one account in the whole detachment needs Google Drive permission: this one. Everybody else — cadre, cadets, the commander — reaches their data through the submission server you deploy in Part 6, and never grants Drive access at all.
>
> Until the app completes Google's verification review, granting that permission means passing an "*unverified app*" screen. On an ordinary Google account that screen has an *Advanced* link and you can go through it. On a university or workplace account, the IT department can set a policy that turns the same screen into a refusal with no way past it — and nobody at the detachment can override it, because the setting does not belong to the detachment.
>
> A new account has no such policy attached, so setup always completes. This costs nothing and takes about two minutes.

1. **Create the account at `accounts.google.com`.**
   Name it after the unit rather than a person — `det025.feedback@gmail.com`, or similar. A personal account takes the data with it when that person moves on.

2. **Record the password where the detachment keeps shared credentials.**
   This account owns every record. Losing it means losing the folder, and the recovery options on a brand-new account are thin. Set up recovery details on a unit phone number or address, not a departing cadet's.

> **You do not create a folder yourself**
>
> Earlier versions of this guide asked you to create a folder in Drive by hand and paste its address into the app. **Do not do that.** The app creates its own folder during setup in Part 4, and a folder you make by hand cannot be used.
>
> The reason is the permission the app asks for. It requests access to *files it creates itself* and nothing else — it cannot see the rest of your Drive and has no way to ask. A folder made by hand was not created by the app, so Google does not show it to the app at all. Creating its own folder is precisely what lets the app avoid asking for access to your entire Google Drive.
>
> Nothing else changes. The folder appears in your Drive, belongs to you, and you can open, move or delete it like any other. You will need its address in Part 6, and the app shows it to you the moment it is created.

2b. Allow the app to reach that Drive

Google requires any app that touches a Drive account to be registered with that account. This is free and takes about ten minutes.

1. **Go to `console.cloud.google.com`**
   signed in as the same Google account.

2. **Create a new project.**
   Name it something recognisable, such as *Det 025 Feedback*. Wait for it to finish creating and make sure it is the selected project in the bar at the top.

3. **Enable the Google Drive API.**
   - Navigate to *APIs & Services → Library*.
   - Search for **Google Drive API** and open it.
   - Click **Enable**.

4. **Configure the OAuth consent screen.**
   - Go to *APIs & Services → OAuth consent screen*. Newer consoles file this under *Google Auth Platform → Branding*.
   - Choose **External**. This is the only option on the new account you made in Part 2a, and it is the correct one — *Internal* exists only for Google Workspace domains, and would restrict sign-in to that domain's own members.
   - Fill in the app name and support email. Nothing else is required.
   - **Leave the publishing status on *Testing*.** Do not click *Publish app* — see the warning below.

5. **Add every Google account that will use the app under *Test users*.**
   - Cadre, administrators and *every cadet*. An account that is not on this list cannot sign in at all. Add whichever address each person will actually sign in with — for most detachments that is the Gmail address the det already mails them at, but a school address works too, because signing in asks for no Drive permission and so does not depend on that school's IT policy.
   - **The account that owns the project is not automatically a test user.** Add it explicitly, like any other.
   - The limit is 100 accounts, which covers a detachment.

6. **Create an OAuth client ID.**
   - Go to *APIs & Services → Credentials → Create Credentials → OAuth client ID*.
   - Application type: **Web application**.
   - Under **Authorised JavaScript origins**, add the exact address the app will be served from — see Part 3 if you do not have it yet. For example `https://det025.github.io`. Include the scheme, exclude any path or trailing slash.
   - Leave *Authorised redirect URIs* empty. The app does not use them.
   - Click **Create** and copy the **Client ID**. It ends in `.apps.googleusercontent.com`.

Do not click "Publish app"

It sounds like the step that makes things work. It is the opposite. 9ThirtyOne asks for Google Drive access on your behalf while you are setting up. Publishing to production without completing Google's verification blocks *everyone*, including you, with "*Access blocked … has not completed the Google verification process*" and no way through. If you have already clicked it, open the consent screen and choose **Back to testing**.

Two different "unverified" screens — they mean different things

"Google hasn't verified this app" with an *Advanced* link is normal and expected. Click *Advanced*, then *Go to … (unsafe)*. Every user sees this, every time, and it cannot be turned off while the app is in Testing.

"Access blocked … has not completed the Google verification process" with *no* Advanced link is a hard refusal. It means either the app was published to production, or that account is not on the test-user list.

Known issue: some accounts cannot be added as test users

Occasionally Google refuses to accept an address on the test-user list, with no useful explanation. The cause is that **some Google accounts are configured never to allow unverified apps** — by enrolment in Google's *Advanced Protection Program*, by being a supervised or Family Link account, or by a workplace or school policy. It is a standing Google-side limitation, not a fault in the app, and it cannot be worked around from this end.

There is no way to tell in advance which accounts these are. If one is refused, that person signs in with a different Google account instead — an ordinary personal Gmail address will work. This is the same reason Part 2a has you create a fresh account for the detachment itself: the one account that *must* work is the one you control from the start.

The Client ID is not a password

It identifies the app to Google and is designed to be public — it is visible in any browser that loads the page. Access is controlled by Google sign-in and by who you shared the folder with, not by keeping this string secret. There is no client *secret* to protect.

If you change where the app is hosted

You must add the new address to **Authorised JavaScript origins** or sign-in will fail with a redirect error. Changes can take a few minutes to take effect.

3 Publish the app

9ThirtyOne is a folder of files with no build step and no backend, so it can be served from anywhere that serves static files.

> **HTTPS is required**
> Offline support, installation to a home screen, and Google sign-in all require a secure connection. Every hosting option below provides HTTPS automatically. Opening the files directly from a hard drive will *not* work.

Option A — free static hosting (recommended)

HOST	NOTES
GitHub Pages	Free. Push the folder to a repository and enable Pages in the settings. Address looks like `https://yourname.github.io/nine-thirty-one`.
Netlify or Cloudflare Pages	Free tier. Drag the folder onto their dashboard — no command line needed.
Your institution's web server	Copy the folder into any directory served over HTTPS.

Option B — run it from one computer

Useful for a trial, or where the detachment cannot publish to the web. Requires Python 3, which is pre-installed on macOS and Linux.

```
cd nine-thirty-one
python3 serve.py              # then open http://localhost:8000

python3 serve.py --https      # needed to reach it from a phone
```

The `--https` option generates a self-signed certificate and prints an address on your local network. Phones will warn that the certificate is untrusted; accept it once. This is fine for testing, but use Option A for real use.

> **Whichever you choose, note the exact address**
> You need it for *Authorised JavaScript origins* in Part 2, step 5.

4 Run the setup wizard

> **If you are signed into more than one Google account, use a clean browser profile**
>
> Setup authenticates *twice* — once so the app can reach Drive, once to say who you are — and each time Google shows an account chooser that defaults to whichever account you signed in with first. Picking the wrong one at either prompt produces a half-configured install that fails confusingly later. A clean browser profile, or a private window signed into only the detachment account, removes the choice entirely. If your Drive address bar shows `/u/1/` or higher, you are not on your default account.

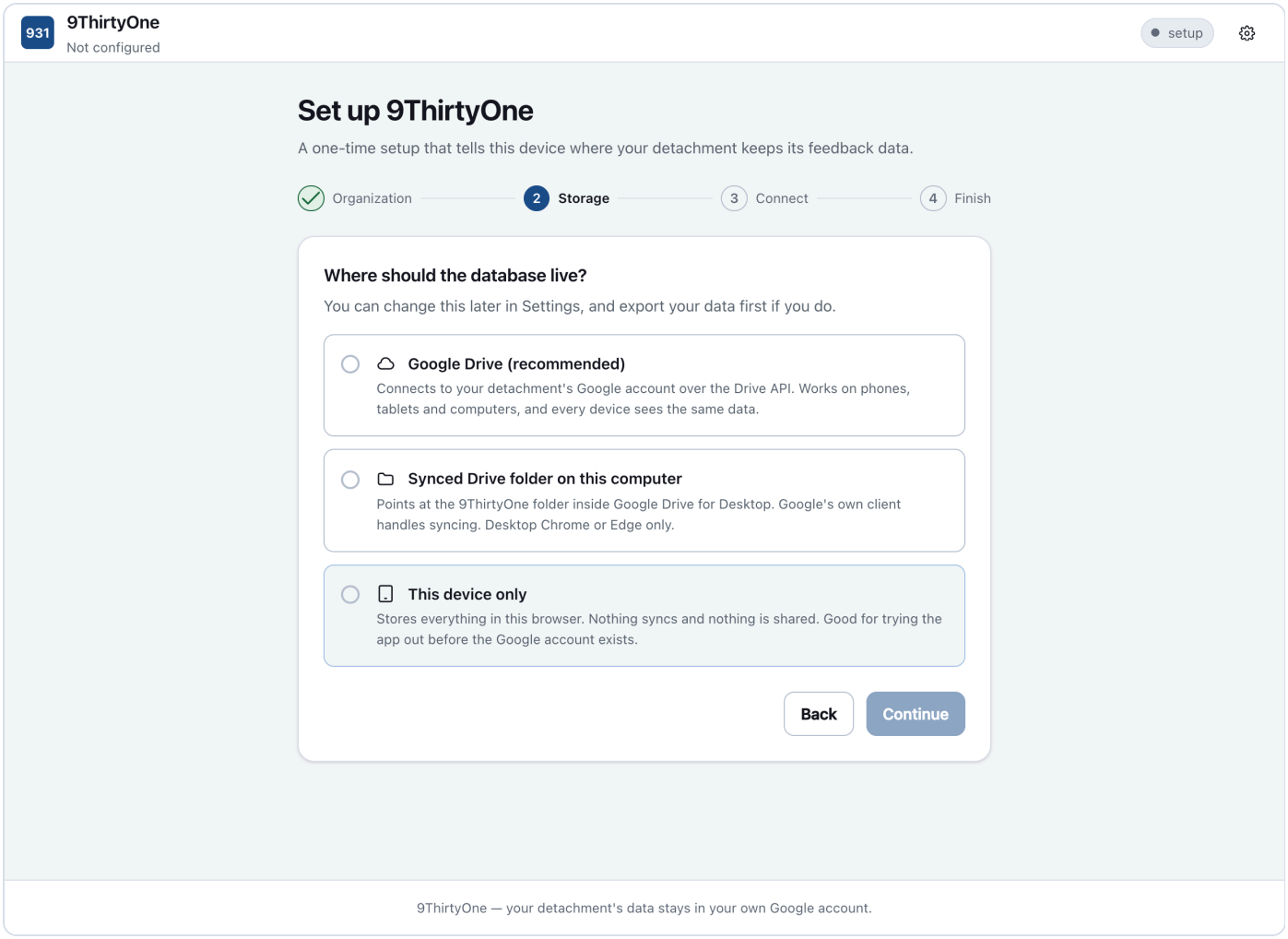
Open the app's address in a browser. On a folder that has never been set up, the wizard starts automatically.

1. **Name the detachment.**
   This appears in the app header and on exported reports.

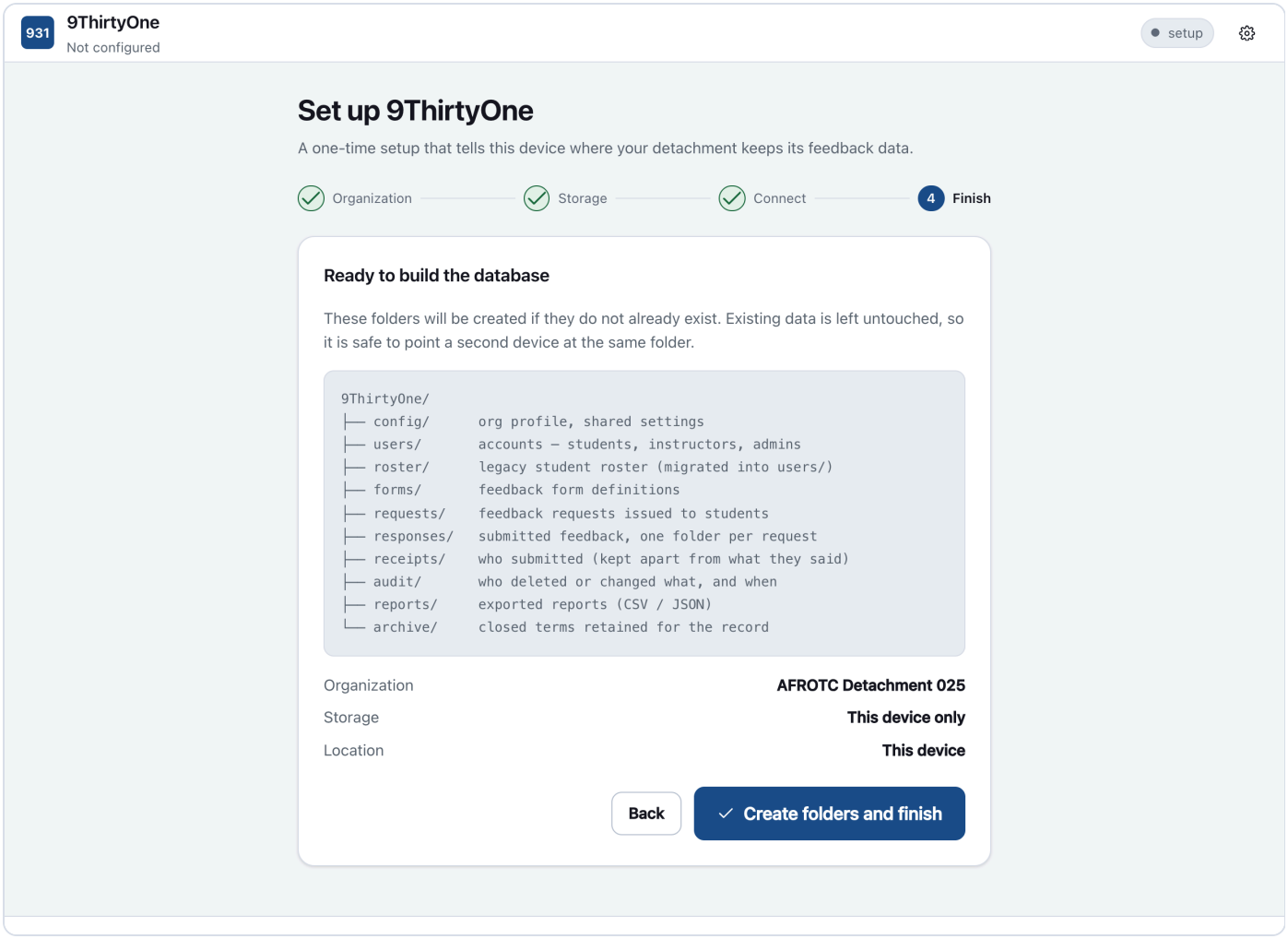
2. **Choose your storage option**
   from Part 1.

3. **Connect.**
   - **Google Drive:** paste the Client ID from Part 2b, then click *Sign in and create the folder*. A Google window opens; sign in as the detachment account and grant access. The app creates its `9ThirtyOne` folder and shows you its address — **copy that address now**, you need it in Part 6. There is no folder link to paste: see Part 2a for why.
   - **Synced folder:** click *Select folder…* and choose the 9ThirtyOne folder inside your Google Drive for Desktop directory.
   - **This device only:** nothing to connect.

4. **Create the folders.**
   The wizard shows exactly what it will create and then builds it. Existing data is never touched, so it is safe to point a second device at the same folder and run the wizard again.



Step 2 of the wizard. Options unavailable on the current device are greyed out with the reason.



Step 4 shows the folder structure before creating it. Each record is a plain JSON file you can read in Drive.

5 Claim the folder

9ThirtyOne issues no passwords. Everyone signs in with the Google account your detachment already mails them at, and a roster inside your Drive folder decides who is allowed in and what they can do. Your first job is to put yourself on that roster.

A folder with an empty roster is claimed by **the first Google account to sign in**, which becomes both a database administrator and an instructor. After that the roster is closed: an account that is not on it is turned away, whoever it belongs to.

1. **Confirm the folder is shared with the right people before you start.**
   Drive sharing is the control that actually matters — see the note below.

2. **From the home screen, open *Database Administration*.**
   It should say this detachment has no roster yet.

3. **Click Sign in with Google**
   and choose the account you want to administer the detachment with.

4. **Confirm you landed in the console**
   and that your email appears on the roster with the *Database admin* and *Instructor* roles.

5. **Add a second administrator.**
   The console will remind you while you are the only one. It is not a formality: a single admin who loses access leaves the roster repairable only by editing `users/users.json` in Drive by hand.

6. **Check the access setting.**
   Go to *Settings* → *Access*. It should read "Google sign-in only".

> **Share the folder before the first sign-in, not after**
> Whoever can reach the app and open the folder can claim an unclaimed one. That is deliberate — it is how a new detachment gets in without a shared password that would be published in a guide like this one. It also means the window between creating the folder and claiming it should be short, and the folder should be shared only with the people who need it.

"Allow signing in with an email instead of Google"

The one setting under *Settings* → *Access*. It exists for an installation with **no Google Client ID** — evaluating the app on a single device, before the Google setup in Part 2 has been done. Without it such an installation is a dead end, because Google cannot sign anybody in when there is no Client ID to sign them in against.

It grants nothing on its own. Whoever signs in that way is checked against the roster exactly like a Google sign-in: an address nobody has added is refused, and the roles that come back are the real ones. It adds a way to say who you are; it is not a way to skip being anybody.

A detachment on Google Drive has a Client ID by definition, so the sign-in screen will not offer it and the setting should stay off. Once your detachment has an administrator account, only an administrator can switch it on — turning it *off* is never restricted.

6 Deploy the submission server

> **This part is no longer optional**
>
> The app asks Google only for access to files it created itself, which means a cadet's phone has no way to reach your detachment's folder — it did not create it. The submission server is what files their feedback for them. **Without it, join links cannot set up anyone else's device.**
>
> Earlier versions could fall back to giving every cadet access to the whole folder. That was the arrangement where any cadet could open Drive and read every response, and it is gone.

Fifteen minutes, and it changes what everything else costs. A small script running inside your own Google account files everyone's feedback for them, so nobody but you needs access to the Drive folder.

> **Without this, everyone who uses the app can read every response**
>
> Google requires *Editor* permission to write a file, and Drive has no write-without-read. So without the server, every cadet who submits feedback can also open the folder and read every response in it, including anonymous ones — and can alter them, or delete the activity log. The anonymity in the app's own screens is real and worth nothing against somebody who opens Drive directly.

With it deployed, three things become true at once:

- **Nobody but you needs Drive access.** Cadets and cadre both reach their data through the server.
- **Nobody sees a Google permission screen asking for their Drive.** The app stops asking for it entirely, which also removes the "*Google hasn't verified this app*" warning your cadets would otherwise meet before they ever saw the product.
- **One submission per cadet becomes a rule rather than a convention**, because the server checks and writes under a lock. Two taps on a bad connection cannot both land.

Deploy it

1. **Open script.google.com**

   signed in as the account that owns the Drive folder. This matters: the script files everything using that account's access.

2. **New project**

   , named `9ThirtyOne Server`. Delete what is there and paste in `tools/proxy/Code.gs` from the app's files.

3. **Fill in the two values at the bottom**

   of the file — your folder ID and your Client ID — then run `setUp` once and clear them out again.

4 **Deploy → New deployment → Web app**

, set to *Execute as: Me* and *Who has access: Anyone*, and copy the address it gives you. It ends in `/exec`.

5 **Paste it into Settings → Submission server**

and press *Save and test*. It checks the deployment is reachable, is the right script, and is configured, so you find out now rather than when a cadet cannot submit.

"Anyone" sounds wrong. It is not.

The address is public, and the script refuses everything that does not carry a valid Google sign-in for somebody on your roster — it checks that with Google on every single request. Access is decided by your roster, not by the address being hard to guess. Cadets are not part of any organisation account, so there is no narrower setting that would work.

Full instructions live with the script

`tools/proxy/README.md` covers each step with screenshots of the settings, what every error message means, and how to redeploy after a change — forgetting to redeploy is the most common reason an edit appears to do nothing.

7 Build the roster

Everyone signs in — cadets included. That is what makes "one submission per cadet" mean something rather than relying on an honour system.

ROLE	CAN DO
Cadet	See feedback assigned to them and submit it once.
Instructor	Create feedback, read responses, run analysis. In *By instructor*, their own results.
Cadre	Everything an instructor can, plus the *Cadre Panel* — the same screen again, holding feedback instructors cannot see. Not a hidden tab: a separate folder they have no access to. In *By instructor*, cadre see every instructor's results, for oversight.
Commander	Sees every space including its own, which nobody else can read. Their own space appears inside the Cadre Panel, marked. In *By instructor* a commander sees everyone, cadre included. **At most two at a time**, so a change of command overlaps.
Database admin	Add and remove people, maintain the database, delete feedback — always with a recorded reason.

Two panels, not one list

The **Instructor Panel** holds the detachment's ordinary feedback. The **Cadre Panel** is the same screen — same tabs, same filters, same form creator — pointed at the restricted folders. Anyone holding cadre can move between them with a button in the top corner, and feedback created from a panel is filed into that panel's space automatically.

They are separate rather than one list with a lock icon for a practical reason: the detachment list can then be read on a shared screen without checking who is standing behind you.

When someone creates feedback they choose which space it belongs to. Detachment feedback is visible to every instructor; cadre-only feedback is not; commander-only feedback is visible to the commander alone. These are separate folders on Drive, enforced by the submission server, so "locked" means locked rather than hidden.

Cadets are asked to fill in feedback from every space — a commander's request is still meant to be answered — and never see anybody's responses, so being asked reveals nothing.

Two commanders, deliberately

Not one, so a change of command overlaps rather than cutting over: the outgoing and incoming commander both hold it during the handover. The space belongs to the detachment, so handing over means moving the designation — no records move, and nothing is lost.

> **Which email address?**
>
> The one your detachment actually mails the cadet at. At a regional detachment that is usually a Gmail address, even for cadets whose school address ends in `.edu` — many colleges run their mail through Google, and the cadet reads it in Gmail. If in doubt, use the address already in your distribution list. It is the address the cadet will click "Sign in with Google" as.

One at a time

1. **Open Database Administration**
   and click *Add person*.
2. **Enter their name and their Google account email.**
3. **Tick the roles they need**
   and choose their AS level.
4. **Save.**
   There is nothing to hand them — they can sign in immediately.

A whole class at once

1. **Prepare a CSV**
   with a `name` column and an `email` column. An optional `class` column sets their AS level.
2. **Click Import roster CSV**
   and choose the file.
3. **Fix anything it could not add.**
   A duplicate address or a typo is listed with the reason. Correct the file and import again — rows already added are skipped rather than duplicated.

Telling them how to get in

Adding someone to the roster does not notify them — this app sends no mail. Once they are on the list, send them the join link from *Invite people* at the top of this same screen. **Copy link** for chat, **Email it** to open a pre-written message, or **Share** on a phone.

For a whole flight at once, **Show QR code** fills the screen with a scannable code. Put it on a projector at the start of a meeting and have cadets point their cameras at it — no app needed, the camera offers to open it. It is the fastest way to set up thirty phones, and nobody types anything.

When someone changes email

Edit their account and change the address. Their submission history follows them: receipts are filed under an internal username such as `alvarez.mia`, which does not change when the email does.

When someone leaves

Untick **Active** rather than deleting them. A deactivated account cannot sign in, but the record stays, so past feedback still makes sense.

Deleting goes further, permanently. Their feedback is kept, but their name is stripped out of it for good: every response they wrote becomes anonymous, and their submission receipts stop identifying them while still counting toward completion. This cannot be undone, and restoring a backup taken afterwards does not bring it back. It is the right choice when somebody asks to be removed; it is not a way to tidy the roster.

> **There is nothing to reset**
>
> If a cadet cannot get in, it is one of three things: the address on the roster is not the one they are signing in with, their account is marked inactive, or they do not have the role for the screen they opened. All three are visible in *Database Administration*. This app has no password to forget.

8 Invite everyone

Everything before this was for your device. Everyone else gets a link.

> **Nobody else runs the setup wizard**
> A join link carries the Client ID, the folder and the submission server, so tapping it sets a device up completely. Cadets sign in with Google and land on their feedback. Nothing to paste, nothing to explain, and no Client ID in the hands of a nineteen-year-old on a phone.

Sending the link

Go to *Database Administration → Invite people*. Three ways out, all the same link:

- **Copy link** for whatever your detachment already uses to talk — group chat, email, the LMS.
- **Email it** opens a pre-written message that explains what to expect.
- **Share** on a phone hands it to the usual share sheet.

A whole flight at once

Show QR code fills the screen with a scannable code. Put it on a projector at the start of a meeting and have cadets point their cameras at it — no app needed, the camera offers to open it. It is the fastest way to set up thirty phones, and nobody types anything.

> **The link is not a password**
> Everything in it is public or an address. The Client ID appears in the page source of every app that uses one, and the folder ID is not a key. Opening the link grants nothing on its own: the roster decides who may sign in, and an address that is not on it is turned away. Post it wherever is convenient.

Adding it to a home screen

DEVICE	HOW
iPhone / iPad	Open the link, tap **Share**, then *Add to Home Screen*. Safari and Chrome both work.
Android	Chrome offers *Install app* in its menu, or prompts on its own.
Mac / Windows	Chrome and Edge show an install icon in the address bar.

> **Add the icon first, then sign in**
> A home-screen app gets its own private storage, separate from the browser it was added from. Open the join link, add the icon, and *then* sign in from the icon — doing it the other way round means signing in twice.

9 Verify and hand over

Work through this before letting cadets use the app for real.

- [ ] **The app opens over HTTPS** at the address you published.
- [ ] **Settings → Access reads "Google sign-in only"** — the email sign-in is off.
- [ ] **At least two people hold the Database admin role**, so losing one account does not lock the detachment out.
- [ ] **The submission server answers** — Settings → Submission server → *Save and test* reports a version.
- [ ] **The Drive folder is shared with nobody but you.** With the server deployed, nobody else needs it.
- [ ] **The connection indicator in the header reads "ready"** on a device other than the one you set up on.
- [ ] **The Drive folder contains the app's folders** — open it in Drive and confirm you can see `config`, `users`, `forms` and the rest.
- [ ] **A test cadet can sign in with their own Google account** and see feedback assigned to them — on their own phone, not yours, reached by tapping a join link.
- [ ] **That cadet was never asked for access to their Google Drive.** If they were, the submission server is not configured on their device — re-send the join link.
- [ ] **A test submission appears** in the Instructor Panel under Responses & Analysis.
- [ ] **You have exported a backup** from Instructor Panel → Database → Export backup.
- [ ] **You have taken one anonymised export** and looked at it, so you know what is in the backup copy you keep off-site.

Moving trial data into Drive

If you evaluated using *This device only*:

1. On the trial device, go to *Instructor Panel → Database → Export backup* and save the JSON file.
2. Go to *Settings → Re-run setup* and connect to Google Drive as in Part 4.
3. Return to *Database → Import backup*, choose the file, and select *Merge*.

Routine maintenance

TASK	HOW OFTEN	WHERE
Export a backup	End of each term	Instructor Panel → Database
Review accounts, deactivate leavers	Start of each term	Database Administration
Rebuild indexes (only if counts look wrong)	Rarely	Database Administration

Troubleshooting

WHAT YOU SEE	WHAT TO DO
"Could not connect" when signing in to Google	The app's address is not listed under *Authorised JavaScript origins*. Add it exactly, including `https://` and with no trailing slash. Wait a few minutes.
Google shows "app is not verified" or blocks sign-in	The OAuth consent screen is set to *External* and this person is not a test user. Add their Google address under *Test users*, or switch to *Internal* if you use Workspace.
The sign-in popup does not open	Popups are blocked for the site. Allow them and try again.
"Access blocked … has not completed the Google verification process", with no *Advanced* link	Either the app was published to production, or this account is not a test user. Open the OAuth consent screen: if it says *In production*, click **Back to testing**. If it says *Testing*, add the account under *Test users* — including the account that owns the project, which is not added automatically.
"Google hasn't verified this app", *with* an *Advanced* link	Expected, not an error. Click *Advanced*, then *Go to … (unsafe)*. Unavoidable while the app is in Testing.
Google refuses to add an address as a test user	A Google-side behaviour with no published fix. Check whether the account holds a role under *IAM & Admin*, whether it is enrolled in Advanced Protection, and try the classic *Credentials* view. If it still will not take, that person needs a different Google account.
Sign-in popup closes immediately, or `origin_mismatch`	The address the app is served from is not listed under *Authorised JavaScript origins*. It must match exactly: scheme included, no path, no trailing slash. Changes can take a few minutes to take effect.
The home-screen icon opens the setup wizard again	Normal. A home-screen app has its own storage, separate from the browser. Run the wizard once inside the icon app and it will not ask again.
Connects, then Drive requests fail with 403	The Google Drive API was never enabled on the project. Part 2, step 2.
"The folder or file was not found"	Almost always a folder created by hand rather than by the app. The app can only reach files it made itself, so a folder you made in Drive is invisible to it however correct the address looks. Run the wizard and let it create one.
Header shows "offline" with a pending count	Normal. Work is saved on the device and sends automatically. Click the pending badge to retry now.
A cadet cannot see a form	Check its status is *Open*, the open date has passed, the due date has not, and the AS level matches theirs. If it targets named cadets, check

	they are on the list.
"Results withheld" instead of analysis	Working as designed. Anonymous results stay hidden until three people have responded, because a single response can be traced to its author by elimination.
"Not on this detachment's roster"	The address they signed in with is not the one on the roster. Ask what Google account they used and compare it with Database Administration — a school address and the Gmail account behind it are two different strings.
Nobody can sign in as an administrator	Open `users/users.json` in Drive, add `"admin"` to the `roles` array of an account you control, and save. This is the recovery path; it is why at least two people should hold the admin role.
Response counts look wrong	Database Administration → *Rebuild indexes*. Counts are a cache and can lag; this rebuilds them from the underlying records.

What this version cannot do yet

Installed and run end to end against real Google Drive. These are its known limits, stated plainly so you can plan around them.

LIMIT	WHAT IT MEANS FOR YOU
100 people per detachment	Google caps an app in Testing at 100 authorised accounts. Comfortable for a detachment; not enough for a wing. Lifting it needs Google's verification process, which is a decision for whoever runs the programme rather than for one detachment.
Some Google accounts refuse unverified apps	A standing Google behaviour, not a fault here. Those people need a different Google account. It is uncommon, and it is why verification matters beyond appearances.
Maintenance runs from the owner's device	Backup, restore, wipe and reindex act on the whole folder, and the submission server deliberately offers no way to do any of them remotely — an address that could empty a detachment's records on request is not one worth having. Sign in on the account that owns the folder.
Written feedback can still identify people	No amount of removing names changes what somebody wrote. A cadet who says they are the only AS400 in their flight has identified themselves. This is true of every feedback system and is worth saying to cadets rather than promising more than can be delivered.
The safety screen matches words, not meaning	It cannot tell "we discussed hazing prevention" from "I was hazed", and a clear screen is not proof nothing was reported. It is a prompt to read something now, never a finding.

Limits that used to be here and no longer are

Earlier versions of this guide warned that everyone who submitted feedback needed Editor access to the Drive folder and could therefore read every response; that every device had to be set up by hand with a Client ID pasted in; and that every user met an alarming Google permission screen. The submission server and join links removed all three. If you are reading an older copy of this guide, replace it.

Reference

What gets created in your folder

```
9ThirtyOne/
├── config/      detachment profile and shared settings
├── users/       the roster — who may sign in, and as what
├── forms/       feedback form definitions
├── requests/    feedback issued to cadets, each with an ID
├── responses/   submitted feedback, one folder per form
├── receipts/    who submitted (kept apart from what they said)
├── audit/       every deletion and roster change, with who did it
├── cadre/       feedback only cadre and the commander can read
├── commander/   feedback only the commander can read
├── reports/     exported reports
└── archive/     closed terms retained for the record
```

Things worth understanding before you field this

Anonymity is real, and it has a limit

On an anonymous form the cadet's sign-in is used to verify them and to write a *receipt*, then discarded. The response record carries no name, and receipts live in a different folder. This is what lets the app say "two cadets still owe feedback" about responses nobody can attribute. The limit: with very few responses, a single answer can be identified by elimination, so anonymous results stay hidden until three people have responded.

The cadre and commander spaces are separate folders

Not hidden screens. `cadre/` and `commander/` are real folders that the submission server will not open for somebody without the role, which is why "locked" here means locked. Feedback cannot be moved between spaces once it exists, because moving it would carry its responses somewhere they were never meant to be readable.

Sign-in is not encryption

Anyone with access to the Drive folder can read every record directly, regardless of what the app shows them. Folder sharing is the real access control. Share it only with people entitled to read all of it.

The app stores no credentials at all — `users/users.json` holds emails and roles, nothing secret. The identity check is Google's. The app validates the sign-in token in the browser, which catches an expired or misdirected one but is not a defence against someone who controls the browser; Drive's own permission check is.

The safety screen is a prompt, not a finding

Written answers are automatically checked for language about hazing, harassment, discrimination, violence, self-harm, substances and integrity. A match means read that response now — it cannot tell "we discussed hazing prevention" from "I was hazed". A clear screen is not proof that nothing was reported. Follow your detachment's own reporting procedures.

Safety alerts override the anonymity threshold

If a flagged answer appears on a form whose results are otherwise withheld, the app tells you it exists and lets you open it behind a warning that doing so may identify the author. A disclosure cannot wait for a third response to arrive.

Getting help

Diagnostics are available at *Settings → Download diagnostics*. The file records the app version, storage configuration and browser, and contains no feedback data — it is safe to share when asking for help.